git と GitHub の基本

GitHubとは？

特別編 GitHub Desktopで体験しよう



2026年10月14日　講師：嘉屋悠月

TODAY ・ 今日のゴール

3つできるようになろう



記録して、戻せる

commit で変更を残し、履歴から前の状態に戻せる



GitHub で共有できる

push で GitHub に送り、pull で他の人の変更を受け取れる



枝を分けて試せる

branch で作業を分け、うまくいったら merge で合流できる

POINT 用語を覚えるのは後から。まず動かして体験してから、知識を定着させる。

TODAY ・ 90分

今日の流れ

01	**git と GitHub を知る** それぞれの役割、ローカルとリモート	8分
02	**手元で記録する** テンプレートから作成 ・ clone ・ commit ・ 履歴から戻す	31分
03	**共有して、枝を分ける** push ・ pull ・ branch ・ merge	26分
04	**4つの違いと Unity の注意** clone ・ commit ・ push ・ pull の整理、.gitignore	6分
05	**ミニミッション** 枝を作って commit し、push して merge する	9分
06	**まとめ・到達チェック** できるようになったことを確かめる	4分
予備	**準備の遅れ・つまずき対応** 各章のあいだに少しずつ使う	3分

SECTION

01

git と GitHub を知る



記録する道具と、置いて共有する場所。
それぞれの役割の違いを知ろう

git は、変更の履歴を残す道具

バージョン管理

ファイルの変更を、いつ・何を変えたか付きで
順番に記録していく仕組み。

ゲームのセーブポイント

失敗しても、前のセーブに戻れる。
git の記録も、同じように使える。

自分のPCの中で動く

git 単体なら、インターネットはいらない。

レポートを直すとき

ファイル名で管理

最終版_v3_本当の最終.docx

git で管理

ファイルは1つ。履歴から、いつでも前の版に戻せる。

? **「最終版_v3」のようなファイル名で保存した経験、ある？**

GitHub は、履歴を置いて共有する場所

インターネット上の置き場

git の履歴をそのまま預けられるサービス。

できること

PCが壊れても残る。
仲間と同じ履歴を共有できる。
ブラウザで中身を見られる。

まとめると

git ＝ 記録する道具
GitHub ＝ 記録を置いて共有する場所

	git	GitHub
なに？	ソフト（道具）	サービス（置き場）
どこで？	自分のPC	インターネット上
役割	**履歴を作る**	**履歴を共有する**

手元と GitHub、2つの場所

ローカル

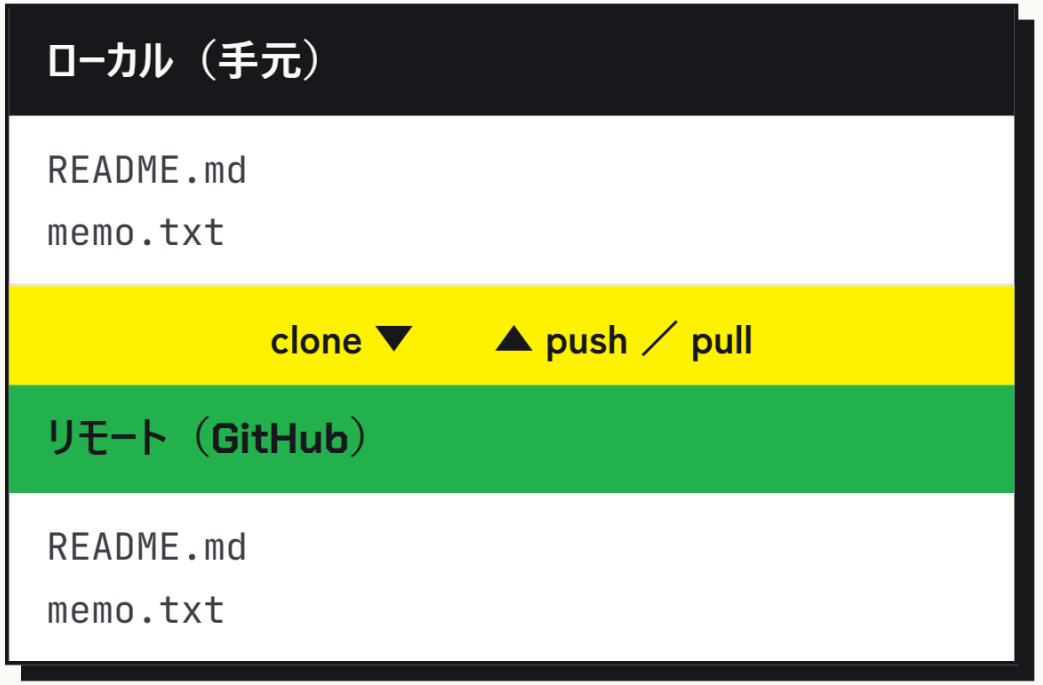
自分のPCの中の、作業フォルダ

リモート

インターネット上の保存場所（GitHub）

リポジトリ

ファイルと履歴を入れた保存場所。
手元にも GitHub にもある。



SECTION

02

手元で記録する



clone で手元に持ってきて、
commit で記録し、履歴から戻そう

自分のリポジトリを作ろう

GitHub のサイト ／ 操作手順

1. 講師が案内するページを開く
github.com/[講師のユーザー名]/fungc-git-practice
2. Use this template ▸ Create a new repository
3. 名前を git-practice にして Create repository

結果

自分のアカウントに、git-practice ができる

中を開くと

README.md ・ memo.txt ・ .gitignore が入っている

? **講師のリポジトリと自分のもの。何が同じで、何が違う？**

GitHub の中身を手元に持ってこよう

GitHub Desktop ／ 操作手順

1. File ▸ Clone repository
2. 自分のリポジトリを選ぶ
3. 保存先を決めて Clone

結果

手元にフォルダができ、中に README.md が入る

GitHub のサイトと比べると

同じファイルが並んでいる

? GitHub のサイトと、手元のフォルダ。中身は同じ？ 違う？

変更を commit で記録しよう

memo.txt ／ 操作手順

1. memo.txt の名前の行に、自分の名前を書いて保存
2. Changes に差分が出たか確認
3. Summary に一言書いて Commit

Changes

追加した1行が、緑色で表示される

History に1件増える

自己紹介を追加

? **commit したのに、GitHub のサイトに出ないのはなぜ？**

commit メッセージは未来の自分へのメモ

⚠ NG　何を直したか分からない

```
aaa ／ 修正
```

数日後に履歴を見ても、
その変更の理由が分からない。

✓ OK　何を変えたかが分かる

```
好きな色を赤に変えた
```

1行で、何を・なぜ変えたかが分かる。
短く、動詞で終える。

履歴から、前の状態に戻そう

History ／ 過去の記録

3　好きな色を赤に変えた

2　自己紹介を追加

1　最初のコミット

2 を選ぶと

その時点のファイルの中身が見られる

Revert（取り消す）

壊した変更を、打ち消す記録を新しく作れる

?　色を赤に変える前の状態に戻すには、どうする？

SECTION

03

共有して、枝を分ける



push と pull で GitHub と行き来し、
branch で作業の枝を分けよう

commit を GitHub に送ろう

GitHub Desktop ／ 操作手順

1. Push origin を押す
2. ブラウザで自分のリポジトリを開く
3. commits で、さっきの commit を探す

push のあと

GitHub のサイトに、commit のメッセージが並ぶ

push の前は

commit は手元にあるだけだった

? **前の「commit したのに出ない」の答えは、何だった？**

GitHub の変更を取り込もう

操作手順

1. GitHub のサイトで memo.txt を開き、鉛筆アイコンで1行書き足して Commit changes（仲間の変更の代わり）
2. Desktop に戻り、Fetch origin を押す
3. Pull origin を押す

pull のあと

手元の memo.txt に、サイトで書いた行が増える

History には

サイトで作った commit が増える

? **push と pull、どちらが「送る」？ どちらが「取り込む」？**

枝を分けて、安全に試そう

GitHub Desktop ／ 操作手順

1. Current Branch ▸ New Branch
2. 名前を color-test にして作成
3. 好きな色を変えて commit する
4. Current Branch を main に戻す

main に戻ると

好きな色は、元の色に戻る

color-test の commit

消えずに、枝に残っている

? **color-test の変更を main に入れるには、どうする？**

枝を main に合流（merge）する

ブランチ（branch）

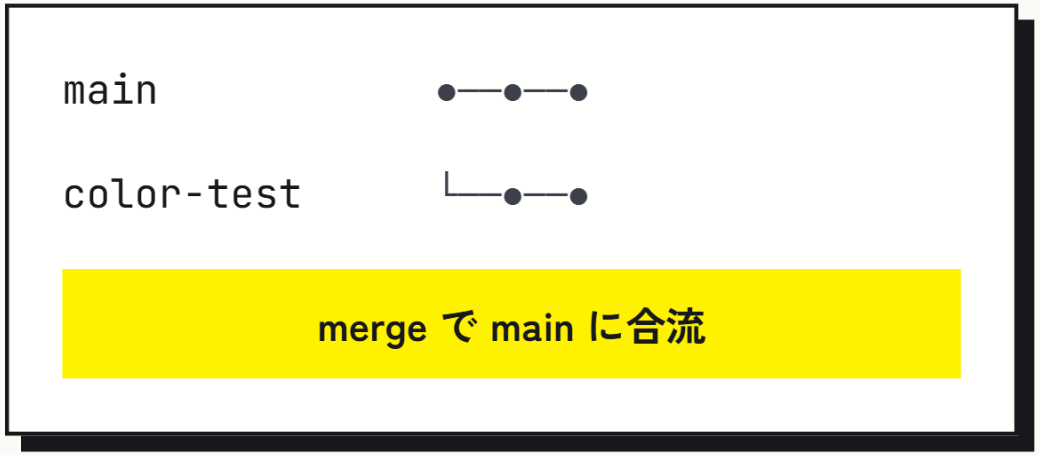
作業を分けて進める「枝」

マージ（merge）

枝の変更を、別の枝に取り込むこと

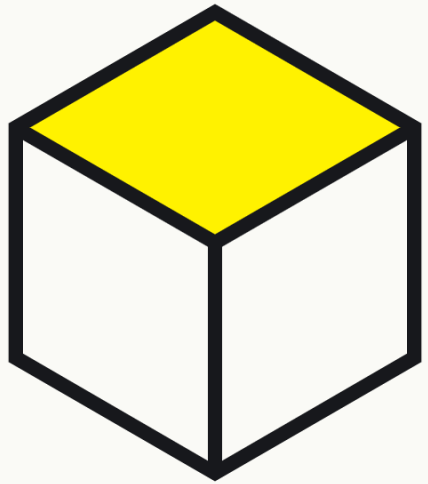
main

基本になる枝。ふつうはここに合流させる



? 全員が main に直接 push すると、何が困る？

POINT ここだけは覚える



clone ＝ 手元に持ってくる
commit ＝ 手元に記録する
push ＝ GitHub に送る
pull ＝ 手元に取り込む

commit だけは、手元の中で終わる。GitHub に届くのは push のあと。

Library と Temp は GitHub に入れない

共有しないもの

Unity が自動で作るフォルダ（Library・Temp など）。
重く、開けばまた作られる。

共有するもの

Assets・Packages・ProjectSettings。
.meta ファイルも一緒に共有する。

.gitignore

共有しないものを書いておくファイル。
練習用リポジトリには入れてある。

Unity プロジェクト	
Assets	共有する
ProjectSettings	共有する
Library	共有しない
Temp	共有しない

EXERCISE

演習

目安時間

9分

ミニミッション：1周してみよう

ゴール　枝を作って commit し、GitHub に push して確かめる

1. my-practice の枝を作って切り替える
2. memo.txt に自分の名前を書いて commit
3. push して、GitHub のサイトで確かめる
4. main に merge する

早く終わった人へ

commit メッセージを3つ書き分けてみる ／ 隣の人の枝を pull してみる

SUMMARY ・ まとめ

3つの道具を覚えた



commit で記録

変更を1つずつ残し、履歴から前の状態に戻せる



push / pull で共有

GitHub と行き来して、チームと同じ状態に



branch で枝分け

枝で試して、merge で main に合流する

NEXT　次回は明日。Unity基礎講座です。

CHECK ・ 到達チェック

できたものに ✓ を付けよう

- [ ] git と GitHub の違いを1文で説明できる
- [ ] clone・commit・push・pull の違いを説明できる
- [ ] リポジトリを clone できる
- [ ] 変更を commit でき、メッセージを1行で書ける
- [ ] push して、GitHub のサイトで確かめられる
- [ ] pull で他の人の変更を取り込める
- [ ] 履歴から前の状態を見て、戻せる
- [ ] ブランチを作って切り替え、main に merge できる
- [ ] Unity で .gitignore が必要な理由を言える